PA4: Occupancy Grid Mapping

Xiaoming Zhao

Screen Recording: [Google Drive Link]

May 12, 2026

Implementation

This programming assignment implemented a ROS 2 node `pa4_occupancy_mapper` subscribes to `/base_scan` and resolves the sensor pose in the `odom` frame via `tf2_ros.Buffer.lookup_transform` at each scan stamp. For every beam it raycasts with Bresenham and applies a log odds Bayesian update along the ray, then publishes the grid as `nav_msgs/OccupancyGrid` on `/map` in the `odom` frame, TRANSIENT_LOCAL. The map grows whenever a beam endpoint leaves the current bounds, no prior knowledge of world extent is required.

Output. The published grid follows the ROS standard, with `-1` unknown; `0` free, cell traversed by a ray and $p <$ `free_thresh` $= 0.35$; `100` occupied, where ray terminated there and $p >$ `occ_thresh` $= 0.65$.

Raycasting. For each beam the world endpoint is computed as $(s_x + r\cos(\theta+\alpha_i),\ s_y + r\sin(\theta+\alpha_i))$ from the sensor pose (s_x, s_y, θ) and beam angle α_i. World coordinates are converted to integer cells, and the line from robot cell to endpoint cell is enumerated with Bresenham algorithm. All cells along the ray receive a free update; the endpoint receives an occupied update if $r < 0.95 \cdot r_{\max}$.

Log odds Bayesian update. Each cell stores its log-odds ℓ; the prior is $\ell_0 = 0$, $P = 0.5$. The recursive Bayes filter in log odds reduces to addition

$$\ell_t(m_i) \;=\; \ell_{t-1}(m_i) \;+\; \log \frac{P(m_i = 1 \mid z_t)}{P(m_i = 0 \mid z_t)}$$

$p_{occ} = 0.7$ and $p_{free} = 0.3$, the per-beam increments are

$$\ell_{occ} = \log(0.7/0.3) = +0.847$$

$$\ell_{free} = \log(0.3/0.7) = -0.847$$

After every scan the array is clipped to $[-\ell_{\text{clamp}}, +\ell_{\text{clamp}}] = [\pm 5.0]$ so a observed wall does not become stuck, clamping caps P at [0.0067, 0.9933]. This update makes the filter resilient to one outlier shifts ℓ by ± 0.85.

Map growth. The grid starts at 200×200 cells centred on `odom` origin. If the sensor pose or any beam endpoint maps to a cell beyond the current bounds, the grid is reallocated with `grow_chunk`= 100 cells added on the affected side. Existing log-odds and arrays are copied into the new offset, and `OccupancyGrid.info.origin` is shifted so all cells remains in their world coordinates.

Design decisions. (1) The map is published in the `odom` frame, not `map`, so it consistent without requiring localization. Log odds are stored as `float32` and converted to thresholded `int8` at publish time; this avoids quantisation errors during accumulation. `beam_skip`= 4 uses every fourth beam, which captures wall feature and decrease computation. Snapshot is saved as PGM + YAML format.

Evaluation and Limitation

Snapshots were recorded after wall-following runs in `2017-02-11-00-31-57.world`. The built maps were resampled onto each ground truth grid using the known robot spawn pose as the `odom`→world offset, figure 1 shows the map overlay. With no external `map` → `odom` correction, as the loop closure, there is no mechanism to close the loop. Also, full coverage would require either a well designed exploration policy or running until every reachable cell was visited, as adding the planning approach. The Bresenham is enumerated at `beam_skip`= 4 and $\leq$ 100-cell rays the cost is $\sim$3 ms per scan, below the 50 ms scan period, but a denser configuration would benefit from a vectorised raycaster.

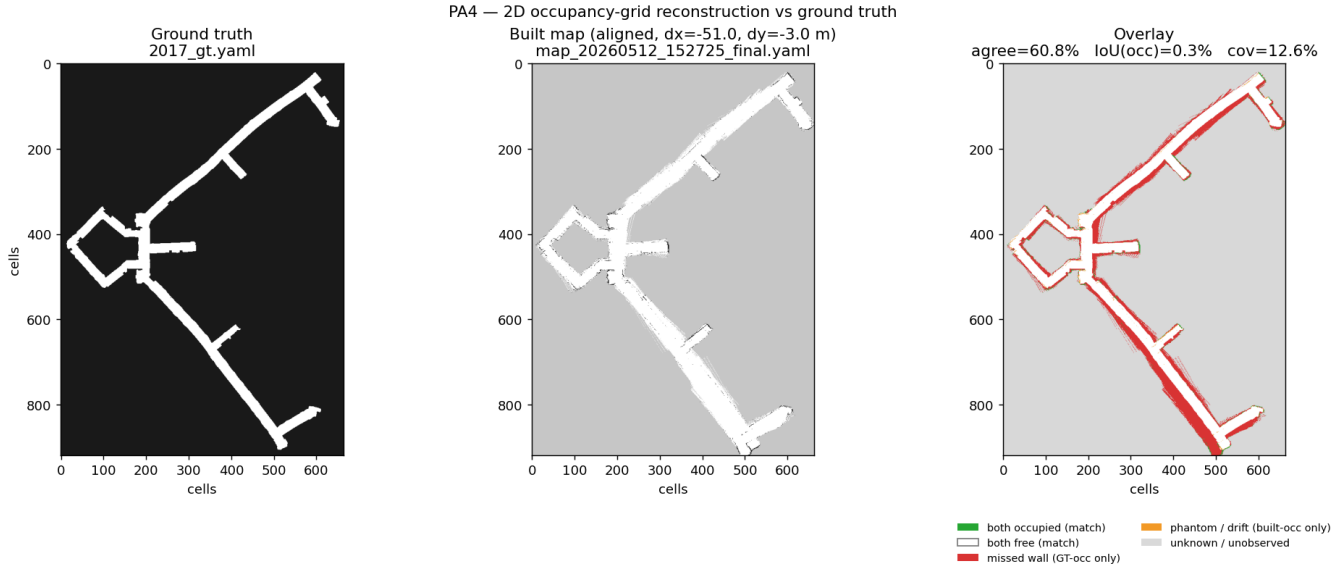

Figure 1: 2D reconstruction of `2017-02-11-00-31-57.world`

Credits

Developed by Xiaoming Zhao, referencing lecture materials, ROS 2 `tf2`/`sensor_msgs`/`nav_msgs` documentation, the `stage_ros2` source, and log-odds occupancy formulation. Debugging, parameter tuning, and report formatting assisted by Generative AI, including Claude and Gemini.